Лабораторна робота № 3

Управляючі конструкції в мові Go. Функції.

Введення та виведення інформації у консоль

Мета роботи: ознайомитися з управляючими конструкціями мови програмування Go, навчитися користуватися функціями введу/виводу та створювати користувацькі функції. Ознайомитися з методами генерування випадкових чисел.

Хід роботи

1. Розробити функцію генерування цілочислової послідовностіпсевдовипадкових значень (за допомогою конгруентного методу) тавиконатиобробку отриманого масиву даних наступним чином (таблиця 3.2): - розрахувати частоту інтервалів появи випадкових величин(інтервалдорівнює 1); - розрахувати статистичну імовірність появи випадковихвеличин; - розрахувати математичне сподівання випадкових величин; - розрахувати дисперсію випадкових величин;

- розрахувати середньоквадратичне відхилення випадковихвеличин.

2. Розробити функцію генерування послідовності псевдовипадковихдійсних значень (за допомогою конгруентного методу) (таблиця 3.2).

Таблиця 3.2 – Вхідні данні

Варіант	Коефіцієнти			Діапазон випадкових величин, n	Довжина послідовності чисел, K
	a	c	m		
2	1664525	1013904223	2^{32}	[0, 150	20000

Результат виконання завдання:

```
package main

import (
        "fmt"
        "math"
)
type LCG struct {
```

```
	a, c, m, current int
}

func NewLCG(seed int) *LCG {
	return &LCG{
		a:	1664525,
		c:	1013904223,
		m:	 1 << 32, // 2^31
		current: seed,
	}
}

func (lcg *LCG) Next() int {
	lcg.current = (lcg.a*lcg.current + lcg.c) % lcg.m
	return lcg.current
}

func (lcg *LCG) NextIntn(n int) int {
	return lcg.Next() % n
}

func (lcg *LCG) NextFloat() float64 {
	return float64(lcg.Next()) / float64(lcg.m)
}

func CalculateStats(frequencies map[int]int, k int) (float64, float64, float64) {
	var expectedValue float64
	var variance float64
```

```
	probabilities := make(map[int]float64)

	for x, freq := range frequencies {
		p := float64(freq) / float64(k) // P = L / K
		probabilities[x] = p
		expectedValue += float64(x) * p // M(X) = Sum(x_i * p_i)
	}

	for x, p := range probabilities {
		diff := float64(x) - expectedValue
		variance += (diff * diff) * p // D(X) = Sum((x_i - M(X))^2 * p_i)
	}

	stdDev := math.Sqrt(variance) // Sigma = sqrt(D(X))

	return expectedValue, variance, stdDev
}

func main() {
	n := 150
	k := 20000
	seed := 1

	lcg := NewLCG(seed)
	frequencies := make(map[int]int)

	// Task 1: Generate Integer Sequence & Frequencies
	for i := 0; i < k; i++ {
		val := lcg.NextIntn(n)
```

```
        frequencies[val]++
    }

    expectedValue, variance, stdDev := CalculateStats(frequencies, k)

    fmt.Printf("--- Integer Sequence Statistics (K=%d, Range=[0, %d)) ---\n", k, n)
    fmt.Printf("Mathematical Expectation M(X): %.4f\n", expectedValue)
    fmt.Printf("Variance D(X):                 %.4f\n", variance)
    fmt.Printf("Standard Deviation σ(X):       %.4f\n\n", stdDev)

    // Task 2: Generate Float Sequence Sample
    lcgFloat := NewLCG(seed)
    fmt.Printf("--- Float Sequence Sample ---\n")
    for i := 0; i < 5; i++ {
        fmt.Printf("Float %d: %.6f\n", i+1, lcgFloat.NextFloat())
    }
}
```

```
:!go run main.go
--- Integer Sequence Statistics (K=20000, Range=[0, 150)) ---
Mathematical Expectation M(X): 74.5108
Variance D(X):                 1882.2633
Standard Deviation σ(X):       43.3851

--- Float Sequence Sample ---
Float 1: 0.236456
Float 2: 0.369271
Float 3: 0.504242
Float 4: 0.704883
Float 5: 0.050544
```

Рис 1. Результат виконання завдання

Висновок: ознайомився з управляючими конструкціями мови програмування Go, навчився користуватися функціями вводу/виводу тас творювати користувацькі функції. Ознайомився з методами генерування випадкових чисел.